

```
#
=====
=====
# Sentiment Analysis Project – Fine-tuned
BERT
#
=====
=====
# Install: pip install transformers torch
scikit-learn pandas
#          matplotlib seaborn nltk
#
=====
=====

import torch
import torch.nn as nn
from torch.utils.data import Dataset,
DataLoader
from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    AdamW,
    get_linear_schedule_with_warmup,
)
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score,
)
```

```

from sklearn.model_selection import
train_test_split
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
from nltk.sentiment.vader import
SentimentIntensityAnalyzer
nltk.download("vader_lexicon", quiet=True)

# — CONFIG


---


DEVICE      = torch.device("cuda" if
torch.cuda.is_available() else "cpu")
MAX_LEN     = 128
BATCH_SIZE  = 16
EPOCHS      = 3
LR          = 2e-5
MODEL_NAME  = "bert-base-uncased"
LABELS      = ["Negative", "Neutral",
"Positive"]
print(f"Using device: {DEVICE}")

# — 1. SAMPLE DATA (replace with your own
CSV) —————
# Expected CSV columns: "text", "label"
(0=Neg, 1=Neu, 2=Pos)

SAMPLE_DATA = {
    "text": [
        "This product is absolutely

```

amazing, I love it!",
"Terrible quality, broke after one day.",
"It's okay, nothing special.",
"Best purchase I've ever made!",
"Very disappointed with the service.",
"Works as expected, no complaints.",
"Exceeded all my expectations, highly recommend!",
"Would not buy again, waste of money.",
"Average product, does the job.",
"Fantastic! Five stars without hesitation.",
"Poor packaging, item arrived damaged.",
"Neither good nor bad, just mediocre.",
"Absolutely love this, worth every penny!",
"Customer support was unhelpful and rude.",
"Decent enough for the price point.",
"Outstanding quality and fast shipping!",
"Completely useless, don't buy this.",
"It's fine, meets basic requirements.",
"Life-changing product, highly recommend!",

```
        "Stopped working after a week, very  
frustrating.",  
    ],  
    "label": [2, 0, 1, 2, 0, 1, 2, 0, 1, 2,  
0, 1, 2, 0, 1, 2, 0, 1, 2, 0],  
}
```

```
df = pd.DataFrame(SAMPLE_DATA)
```

```
# — Or load your own CSV —  
# df = pd.read_csv("your_data.csv") # must  
have "text" and "label" columns
```

```
print(f"\nDataset shape: {df.shape}")  
print(df["label"].value_counts().rename({0:  
"Negative",1:"Neutral",2:"Positive"}))
```

```
# — 2. DATASET CLASS
```

```
class SentimentDataset(Dataset):  
    def __init__(self, texts, labels,  
tokenizer, max_len):  
        self.texts      = texts  
        self.labels      = labels  
        self.tokenizer   = tokenizer  
        self.max_len     = max_len  
  
    def __len__(self):  
        return len(self.texts)  
  
    def __getitem__(self, idx):
```

```

        encoding = self.tokenizer(
            self.texts[idx],
            max_length=self.max_len,
            padding="max_length",
            truncation=True,
            return_tensors="pt",
        )
        return {
            "input_ids":
encoding["input_ids"].squeeze(),
            "attention_mask":
encoding["attention_mask"].squeeze(),
            "labels":
torch.tensor(self.labels[idx],
dtype=torch.long),
        }

```

— 3. MODEL

```

class SentimentBERT(nn.Module):
    def __init__(self, num_classes=3,
dropout=0.1):
        super().__init__()
        self.bert =
BertForSequenceClassification.from_pretrain
ed(
            MODEL_NAME,
            num_labels=num_classes,
            hidden_dropout_prob=dropout,

attention_probs_dropout_prob=dropout,

```

```

    )

    def forward(self, input_ids,
attention_mask, labels=None):
        return self.bert(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels,
        )

```

— 4. TRAIN / EVAL FUNCTIONS

```

def train_epoch(model, loader, optimizer,
scheduler):
    model.train()
    total_loss, correct, total = 0, 0, 0
    for batch in loader:
        optimizer.zero_grad()
        input_ids =
batch["input_ids"].to(DEVICE)
        attention_mask =
batch["attention_mask"].to(DEVICE)
        labels =
batch["labels"].to(DEVICE)

        outputs = model(input_ids,
attention_mask, labels=labels)
        loss = outputs.loss
        logits = outputs.logits

        loss.backward()

```

```

torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    scheduler.step()

    total_loss += loss.item()
    correct    += (logits.argmax(-1) ==
labels).sum().item()
    total      += labels.size(0)

    return total_loss / len(loader),
correct / total

def eval_epoch(model, loader):
    model.eval()
    total_loss, correct, total = 0, 0, 0
    all_preds, all_labels = [], []

    with torch.no_grad():
        for batch in loader:
            input_ids      =
batch["input_ids"].to(DEVICE)
            attention_mask =
batch["attention_mask"].to(DEVICE)
            labels         =
batch["labels"].to(DEVICE)

            outputs = model(input_ids,
attention_mask, labels=labels)
            logits  = outputs.logits

            total_loss +=
outputs.loss.item()

```

```

        preds      = logits.argmax(-1)
        correct    += (preds ==
labels).sum().item()
        total      += labels.size(0)

all_preds.extend(preds.cpu().numpy())

all_labels.extend(labels.cpu().numpy())

    return total_loss / len(loader),
correct / total, all_preds, all_labels

```

— 5. MAIN TRAINING LOOP

```

tokenizer =
BertTokenizer.from_pretrained(MODEL_NAME)

train_texts, val_texts, train_labels,
val_labels = train_test_split(
    df["text"].tolist(),
    df["label"].tolist(),
    test_size=0.2, random_state=42,
    stratify=df["label"]
)

train_dataset =
SentimentDataset(train_texts, train_labels,
tokenizer, MAX_LEN)
val_dataset    =
SentimentDataset(val_texts,    val_labels,
tokenizer, MAX_LEN)

```

```

train_loader = DataLoader(train_dataset,
batch_size=BATCH_SIZE, shuffle=True)
val_loader   = DataLoader(val_dataset,
batch_size=BATCH_SIZE)

model        = SentimentBERT().to(DEVICE)
optimizer    = AdamW(model.parameters(),
lr=LR, weight_decay=0.01)
scheduler    =
get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=len(train_loader),
    num_training_steps=len(train_loader) *
EPOCHS,
)

history = {"train_loss":[], "val_loss":[],
"train_acc":[], "val_acc":[]}

print("\n— Training —")
for epoch in range(1, EPOCHS + 1):
    tr_loss, tr_acc = train_epoch(model,
train_loader, optimizer, scheduler)
    vl_loss, vl_acc, preds, labels_true =
eval_epoch(model, val_loader)

    history["train_loss"].append(tr_loss)
    history["val_loss"].append(vl_loss)
    history["train_acc"].append(tr_acc)
    history["val_acc"].append(vl_acc)

    print(f"Epoch {epoch}/{EPOCHS} | "
          f"Train Loss: {tr_loss:.4f} Acc:
{tr_acc:.4f} | ")

```

```
        f"Val Loss: {vl_loss:.4f}  Acc: {vl_acc:.4f}")
```

```
print("\n— Classification Report —")
print(classification_report(labels_true,
preds, target_names=LABELS))
```

```
# — 6. PLOTS
```

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
fig.suptitle("SentimentBERT – Training Results", fontsize=14, fontweight="bold")
```

```
# Loss curve
axes[0].plot(history["train_loss"],
label="Train", marker="o")
axes[0].plot(history["val_loss"],
label="Val", marker="o", linestyle="--")
axes[0].set_title("Loss");
axes[0].set_xlabel("Epoch")
axes[0].legend(); axes[0].grid(alpha=0.3)
```

```
# Accuracy curve
axes[1].plot(history["train_acc"],
label="Train", marker="o", color="green")
axes[1].plot(history["val_acc"],
label="Val", marker="o", linestyle="--",
color="orange")
axes[1].set_title("Accuracy");
axes[1].set_xlabel("Epoch")
```

```

axes[1].legend(); axes[1].grid(alpha=0.3)

# Confusion matrix
cm = confusion_matrix(labels_true, preds)
sns.heatmap(cm, annot=True, fmt="d",
            cmap="Blues",
            xticklabels=LABELS,
            yticklabels=LABELS, ax=axes[2])
axes[2].set_title("Confusion Matrix")
axes[2].set_ylabel("True");
axes[2].set_xlabel("Predicted")

plt.tight_layout()
plt.savefig("training_results.png",
            dpi=150)
plt.show()
print("Plot saved → training_results.png")

```

— 7. INFERENCE FUNCTION

```

def predict(text: str) -> dict:
    """Run inference on a single string."""
    model.eval()
    encoding = tokenizer(
        text,
        max_length=MAX_LEN,
        padding="max_length",
        truncation=True,
        return_tensors="pt",
    )
    with torch.no_grad():
        logits = model(

```

```

encoding["input_ids"].to(DEVICE),

encoding["attention_mask"].to(DEVICE),
    ).logits

    probs      = torch.softmax(logits,
dim=-1).squeeze().cpu().numpy()
    pred_idx   = probs.argmax()

    return {
        "text":      text,
        "label":     LABELS[pred_idx],
        "confidence":
float(probs[pred_idx]),
        "scores": {
            "Negative": float(probs[0]),
            "Neutral":  float(probs[1]),
            "Positive": float(probs[2]),
        },
    }

```

— 8. QUICK TEST

```

print("\n— Inference Examples —")
test_sentences = [
    "This is the best thing I have ever
bought!",
    "Absolutely terrible, do not waste your
money.",
    "It's alright, does what it says on the

```

```

box.",
]

for sentence in test_sentences:
    result = predict(sentence)
    print(f"\nText      :
{result['text']}")
    print(f"Prediction: {result['label']}
({result['confidence']:.2%})")
    print(f"Scores      :
{result['scores']}")

# — 9. SAVE MODEL


---



torch.save(model.state_dict(),
"sentiment_bert.pt")
tokenizer.save_pretrained("tokenizer/")
print("\nModel saved → sentiment_bert.pt")
print("Tokenizer saved → tokenizer/")

# — 10. RELOAD & USE LATER


---



#
# model = SentimentBERT().to(DEVICE)
#
model.load_state_dict(torch.load("sentiment
_bert.pt"))
# tokenizer =
BertTokenizer.from_pretrained("tokenizer/")

```

